THEORY OF COMPUTATION (TOC) — COURSE PROJECT

AutomataLab

Interactive Regular Expression to DFA Converter & Visualizer

Topic 5: Regular Expression to DFA Converter with GUI

Subject:	Theory of Computation (CST 301)
Department:	Computer Science & Engineering
Project Type:	4-Member Team Project
Live Demo:	https://automatalab.netlify.app/
Source Code:	github.com/nikhilbijuk/automatalab
Architecture:	Vanilla JavaScript (ES Modules), Zero Dependencies

Table of Contents

1. **PART 1: PROJECT REPORT & SYNOPSIS**
    - 1.1 Abstract & Project Overview
    - 1.2 Problem Statement & Objectives
    - 1.3 Mathematical Formulations (5-tuple representation)
    - 1.4 Core Algorithms & Pseudocode
        - Shunting-Yard Infix to Postfix with Explicit Concatenation
        - Thompson's Construction (Postfix to ε-NFA)
        - Subset Construction with ε-Closure (ε-NFA to DFA)
        - DFA String Acceptance Simulator
    - 1.5 System Architecture & Flowchart
    - 1.6 Verification & Test Cases (Automated Suite)
2. **PART 2: VIVA VOCE & PRESENTATION DEFENCE GUIDE**
    - 2.1 Foundational Automata Theory Questions (Q1 – Q6)
    - 2.2 Thompson's Construction Deep Dive (Q7 – Q12)
    - 2.3 Subset Construction & State Explosion (Q13 – Q18)
    - 2.4 String Simulation & Dead State Handling (Q19 – Q22)
    - 2.5 Examiner Traps & Advanced Edge Cases (Q23 – Q28)
3. **PART 3: ADVANCED FEATURES, UI ENHANCEMENTS & TEAM ROLES**
    - 3.1 DFA Minimization (Hopcroft's Algorithm Formulation)
    - 3.2 Graph Rendering Architecture (SVG / Canvas Layout)
    - 3.3 Deployment Pipeline (GitHub & Netlify CI/CD)
    - 3.4 4-Member Work Division & Viva Role Allocation

Part 1: Project Report & Synopsis

1.1 Abstract

AutomataLab is an interactive educational and analytical software tool designed to bridge theoretical formal language concepts with visual, algorithmic execution. Given an arbitrary regular expression over alphanumeric alphabets with union (`|`), concatenation, Kleene star (`*`), plus (`+`), and optional (`?`) operators, the system validates syntax, converts the expression to postfix notation, constructs an equivalent ε-NFA via Thompson's Construction, and transforms the non-deterministic machine into a Deterministic Finite Automaton (DFA) using Subset Construction and ε-closures. The platform features an interactive GUI that allows step-by-step state inspection, transition table rendering, and step-by-step string acceptance simulation. It runs entirely client-side with zero external frameworks or backend dependencies.

1.2 Problem Statement & Objectives

In academic study of Theory of Computation (TOC), students often find manual translation between regular expressions, non-deterministic finite automata, and deterministic finite automata tedious and error-prone. Standard tools either use opaque black-box regex libraries (which hide theoretical automata) or generate static images without explaining how intermediate states arise.

Core Objectives:

- Implement Thompson's Algorithm from scratch to construct inductive ε-NFA fragments.
- Implement Subset Construction and ε-closure to determinize the NFA into a DFA.
- Construct and display a complete DFA transition table including dead/trap states.
- Provide an interactive string tester that visually traces transitions and outputs `Accepted` or `Rejected`.
- Ensure 100% offline portability and instant cloud deployment.

1.3 Mathematical Formulations

Formally, both the generated NFA and DFA are 5-tuples:

Automaton Definition: $M = (Q, \Sigma, \delta, q_0, F)$

- Q : Finite set of states.
- Σ : Finite set of input symbols (the alphabet).
- δ : Transition function:
  - For ε-NFA: $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$
  - For DFA: $\delta: Q \times \Sigma \to Q$

- $q_0 \in Q$: Initial / start state.
- $F \subseteq Q$: Set of final / accepting states.

1.4 Core Algorithms & Pseudocode

A. Infix to Postfix with Explicit Concatenation

Standard regular expressions use implicit concatenation (e.g., `ab` means $a \cdot b$). Before executing the Shunting-Yard algorithm, an explicit concatenation operator ($\cdot$) is inserted between adjacent symbols where appropriate.

```
Algorithm: InsertExplicitConcatenation(regex):
  output = ""
  for i from 0 to length(regex) - 1:
    c1 = regex[i]
    output.append(c1)
    if i + 1 < length(regex):
      c2 = regex[i + 1]
      if (isAlphanumeric(c1) or c1 in {')', '*', '+', '?'}) and
         (isAlphanumeric(c2) or c2 == '('):
        output.append('·')
  return output
```

B. Thompson's Construction (ε-NFA)

Processes the postfix regex linearly using a stack of NFA fragments. Each fragment has a single start state and a single accept state.

- **Base Case (Symbol `a`):** Creates state s_0 and s_1 with transition $s_0 \xrightarrow{a} s_1$.
- **Base Case (ε):** Creates transition $s_0 \xrightarrow{\varepsilon} s_1$.
- **Union (`A | B`):** New start state with ε-transitions to `A.start` and `B.start`; `A.accept` and `B.accept` have ε-transitions to a new final state.
- **Concatenation ($A \cdot B$):** Merges or connects $A.accept \xrightarrow{\varepsilon} B.start$.
- **Kleene Star (`A*`):** Adds ε-loops from $A.accept \to A.start$ and bypass from new start to new final state.

C. Subset Construction & ε-Closure

Converts the ε-NFA into an equivalent DFA where each DFA state corresponds to a subset of NFA states.

```
Algorithm: SubsetConstruction(NFA):
  DFA_States = []
  q0_set = epsilon_closure({NFA.start})
  add q0_set to DFA_States as unmarked

  while exists unmarked state T in DFA_States:
    mark T
    for each symbol a in Alphabet:
      U = epsilon_closure(move(T, a))
      if U is not empty:
        if U not in DFA_States:
          add U to DFA_States as unmarked
        DFA_Transitions[T, a] = U
      else:
        DFA_Transitions[T, a] = DEAD_STATE

  Mark any DFA state containing NFA.accept as an accepting state.
```

1.5 Verification & Automated Test Suite

Regex	Test Input	Expected	Outcome	Theoretical Purpose
`(a|b)*abb`	`aabb`	**Accepted**	PASS	Tests union, Kleene star, and trailing pattern
`(a|b)*abb`	`abab`	**Rejected**	PASS	Tests rejecting invalid suffix
`a*`	`""` (empty string)	**Accepted**	PASS	Verifies ε-acceptance of Kleene star
`a+b?`	`a`	**Accepted**	PASS	Verifies plus (1+) and optional (0) operators
`(a|b)*`	`baab`	**Accepted**	PASS	Universal language over {a,b}

Part 2: Viva Voce & Presentation Defence Guide

2.1 Foundational Automata Theory Questions

Q1. What is the fundamental difference between an NFA and a DFA?

In a DFA, for every state and every alphabet symbol, there is **exactly one** deterministic next state. No ε-transitions are permitted. In an NFA, for a given state and symbol, there can be **zero, one, or multiple** transitions, as well as ε (spontaneous) transitions without consuming input.

Q2. Are NFAs more powerful than DFAs in terms of language recognition?

No. Both NFAs and DFAs recognize exactly the same class of languages: *Regular Languages*. By the Subset Construction theorem (Rabin & Scott), every NFA can be converted into an equivalent DFA.

Q3. What is an ε-closure?

The ε-closure of a state `s` (denoted `ε-closure(s)`) is the set of all states reachable from `s` by taking zero or more ε-transitions without consuming any input character.

2.2 Thompson's Construction Deep Dive

Q4. Why does Thompson's construction introduce so many ε-transitions?

Thompson's algorithm is strictly compositional and inductive. It builds complex automata by plugging smaller sub-automata into fixed templates (union, concatenation, star). The ε-transitions guarantee that fragments can be joined without state conflicts or modifying the internal transition logic of sub-expressions.

Q5. For a regular expression of length `n`, what is the upper bound on the number of states in a Thompson ε-NFA?

The number of states is linearly bounded: at most `2n` states, and at most `4n` transitions. This linear guarantee makes Thompson's algorithm computationally optimal for NFA creation.

2.3 Subset Construction & State Explosion

Q6. What is the theoretical maximum number of states in a DFA converted from an NFA with `N` states?

The maximum number is 2^N (the power set of NFA states). This is known as *state explosion*. However, in practical regular expressions, the number of reachable states is typically close to N.

Q7. How do you determine whether a DFA state is an accepting (final) state?

A DFA state $S = \{s_1, s_2, ..., s_k\}$ is an accepting state if and only if **at least one** NFA state $s_i \in S$ is an accepting state of the original ε-NFA.

Q8. What is a Dead / Trap state in a DFA?

A dead state (often represented as ∅) is a non-accepting state from which no accepting state can ever be reached. Once entered, all transitions loop back to itself. In our transition table, transitions on undefined inputs lead to this trap state.

2.4 Examiner Traps & Viva Defense Tips

Q9. Examiner: "Is the DFA generated by your project minimal?"

Model Answer: *"No, sir/ma'am. The DFA generated by Subset Construction is equivalent to the regular expression, but not necessarily minimal. Minimization requires a subsequent partitioning algorithm, such as Hopcroft's or the Table-Filling (Myhill-Nerode) algorithm. In our architecture, the subset-construction DFA remains directly traceable to NFA state sets."*

Q10. Examiner: "Did you use a built-in regex library to test strings?"

Model Answer: *"Absolutely not. All tokenization, Shunting-Yard postfix conversion, Thompson graph construction, ε-closures, and DFA simulation are implemented strictly from scratch in `src/automata.js` using fundamental discrete math structures."*

Part 3: Advanced Features, UI & Team Roles

3.1 DFA Minimization (Hopcroft's Algorithm Formulation)

To reduce the DFA to its minimum number of states, states are partitioned into equivalence classes:

1. **Initial Partition:** `P = { F, Q \ F }` (Accepting states vs Non-accepting states).
2. **Refinement:** For each set `B ∈ P` and each alphabet symbol `a ∈ Σ`, split `B` into B_1 and B_2 if states in `B` transition into different partitions on symbol `a`.
3. **Termination:** Repeat until no partition can be further split. Merge states within the same group into a single minimal state.

3.2 Software Design & UI Architecture

- **Zero-Dependency Design:** The application uses native JavaScript ES Modules, semantic HTML5, and CSS Grid/Flexbox without bulky node bundles or build steps.
- **Graph Visualization:** Nodes and transitions are laid out on responsive visual canvases with designated start arrows (`→`) and double-bordered final states.
- **Real-Time Simulation:** As each character of a test string is processed, the active DFA node pulses green, while the trace log records the exact state trajectory (e.g., `q0 →[a]→ q1 →[b]→ q2 (Accepted)`).

3.3 Live Production & Deployment

Platform	URL	Purpose
Netlify (Live Site)	https://automatalab.netlify.app/	Instant public demonstration on any phone/laptop during viva
GitHub (Source)	github.com/nikhilbijuk/automatalab	Repository, version control history, automated tests

3.4 Suggested 4-Member Work Division

Team Member	Assigned Module	Key Deliverables & Viva Talking Points
Member 1 (Lead / Parser)	Regex Parsing & Tokenizer	Validation, handling operator precedence, inserting explicit concatenation, Shunting-Yard infix-to-postfix conversion.
Member 2 (Automata Core)	Thompson's Construction	Converting postfix expressions into ε-NFA fragments, managing state IDs, handling union, star, plus, and optional operators.
Member 3 (DFA Engine)	Subset Construction & ε-Closure	Computing ε-closure sets, mapping NFA state sets to DFA states, detecting accepting states, generating the DFA transition table.
Member 4 (UI & Simulator)	Frontend & String Simulation	Graph rendering, step-by-step UI controls, string acceptance trace animation, test suite integration.